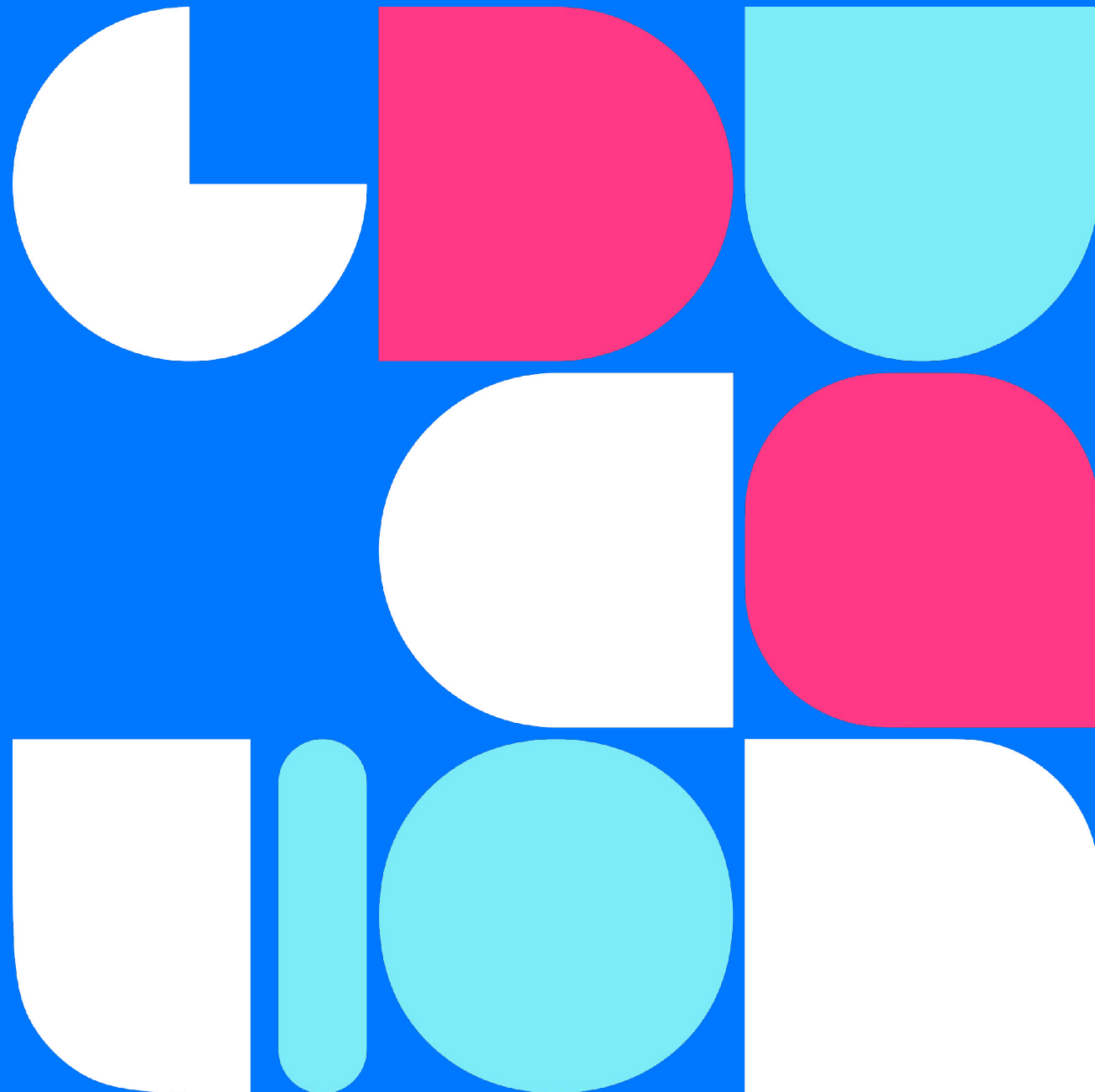




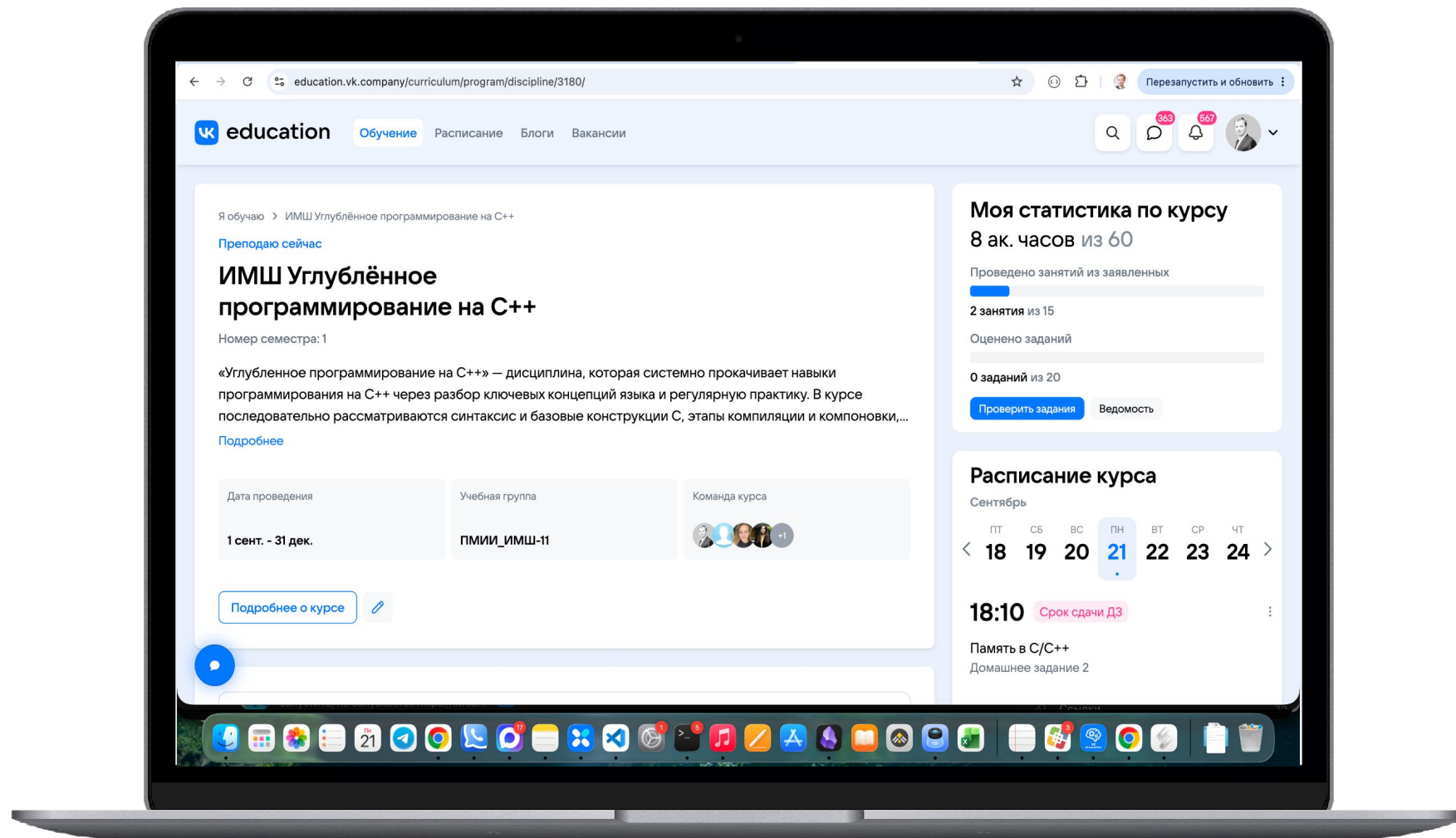
Работа с файлами

Кухтичев Антон



Напоминание отметиться на портале

и оставить отзыв
после лекции



Содержание занятия

- Файлы в Unix, дескрипторы и поток в C
- Чтение, запись, буферы и ошибки
- Пути и границы стандартного C
- Сравнение с C++ и RAII
- `std::filesystem`, практика и упражнения

Файлы в Unix, дескрипторы и поток в C



Что такое файл в Unix

В Unix **файл** — это общий интерфейс к данным и устройствам: многое выглядит как последовательность байтов.

Обычный файл, каталог, канал, сокет и терминал часто доступны через похожую модель ввода-вывода.

Идея Unix звучит как 'everything is a file', хотя типы объектов внутри всё же различаются. Программе важно не столько 'что это физически', сколько 'можно ли это читать, писать или опрашивать'.

Из-за этого одни и те же системные вызовы и похожие абстракции работают для разных источников данных.

Файловый дескриптор

Что это такое	Зачем он нужен
маленькое целое число, например 0, 1, 2, 3	через него процесс обращается к открытому файлу или устройству
идентификатор открытого ресурса внутри процесса	ядро хранит таблицы открытых объектов и прав доступа
выдаётся ядром после успешного open	на дескрипторах работают read, write, close, dup

Стандартные дескрипторы и связь с stdio (1)

Даже до открытия собственных файлов у процесса уже обычно есть три стандартных дескриптора.

- 0 -> stdin (стандартный ввод)
- 1 -> stdout (стандартный вывод)
- 2 -> stderr (стандартный поток ошибок)

```
int fd = open("report.txt", O_RDONLY);  
FILE *f = fopen("report.txt", "r");
```

Стандартные дескрипторы и связь с stdio (2)

- open возвращает файловый дескриптор уровня системных вызовов.
- fopen создаёт более удобную библиотечную обёртку FILE* поверх дескриптора.
- stdio добавляет буферизацию и форматированный ввод-вывод поверх низкого уровня.

Что такое файловый поток

Файл для программы — это поток байтов, через который мы последовательно читаем данные или записываем их.

`stdin`, `stdout` и файл используют похожую идею: есть источник или приёмник данных.

В C эта абстракция представлена типом `FILE` и функциями из `<stdio.h>`.

Программа обычно проходит четыре шага: открыть -> использовать -> проверить -> закрыть.

Ошибки чаще всего возникают не в формулах, а в пропущенной проверке или неверном режиме.

fopen и режимы открытия

Режим говорит, что программа собирается делать с файлом и можно ли потерять старое содержимое.

r

читать только существующий файл

w

создать новый или полностью перезаписать старый

a

добавлять данные в конец, не стирая прежнее

rb / wb

читать и писать бинарные данные

```
FILE *f = fopen("notes.txt", "w");  
if (f == NULL) {  
    perror("notes.txt");  
    return 1;  
}
```

Запись текстовых данных в C

Для простых текстовых файлов обычно достаточно fprintf или fputs.

```
#include <stdio.h>

FILE *f = fopen("report.txt", "w");
if (f == NULL) {
    perror("report.txt");
    return 1;
}
fprintf(f, "User: %s\n", "Alice");
fprintf(f, "Score: %d\n", 42);
fclose(f);
```

fprintf похож на printf, только пишет в файл, а не в консоль.

fputs удобен, когда форматировать ничего не нужно и уже есть готовая строка.

Данные могут лежать в буфере до fclose, поэтому закрытие файла — часть записи.

Чтение текста через fgets

fgets обычно безопаснее и предсказуемее, чем сложные шаблоны fscanff.

```
char line[256];
FILE *f = fopen("report.txt", "r");
if (f == NULL) {
    perror("report.txt");
    return 1;
}
while (fgets(line, sizeof(line), f) != NULL) {
    printf("%s", line);
}
fclose(f);
```

fgets читает не больше размера буфера и помогает избежать переполнения.

После чтения строку можно дальше разбирать отдельно: числа, слова, CSV.

Такой подход часто проще отлаживать, чем сложный формат сразу в fscanff.

Бинарные файлы: fread и fwrite

Когда работаем не с текстом, а с байтами, нужны fread и fwrite.

```
int values[4] = {10, 20, 30, 40};  
FILE *out = fopen("data.bin", "wb");  
fwrite(values, sizeof(int), 4, out);  
fclose(out);  
  
FILE *in = fopen("data.bin", "rb");  
int x[4];  
fread(x, sizeof(int), 4, in);  
fclose(in);
```

fwrite пишет блоки памяти: указатель, размер элемента, число элементов.

fread возвращает, сколько элементов удалось прочитать на самом деле.

Бинарный формат компактнее, но хуже читается человеком и зависит от представления данных.

Буферизация и почему `fclose` важен

Стандартная библиотека старается работать быстрее и не пишет каждый символ на диск отдельно.

Данные часто сначала попадают в буфер, а уже потом записываются в файл.

`fflush` принудительно выталкивает буфер, когда это действительно нужно.

`fclose` не только освобождает ресурс, но и завершает запись оставшихся данных.

При аварийном завершении программы часть буфера может потеряться.

Ошибки чтения и `errno`

Открыть файл — ещё не вся история: ошибки могут появиться и во время последующих операций.

- `perror` печатает последнее системное описание ошибки.
- `feof` сообщает о достижении конца файла, а `ferror` — именно об ошибке.
- `errno` помогает понять причину системного сбоя глубже.

Сообщение об ошибке должно быть понятным не только машине, но и человеку.

Позиция в файле: fseek, ftell, rewind

Иногда нужно не просто читать по порядку, а перейти в другое место файла.

```
FILE *f = fopen("data.bin", "rb");  
fseek(f, 0, SEEK_END);  
long size = ftell(f);  
rewind(f);  
printf("size = %ld\n", size);  
fclose(f);
```

`fseek` перемещает текущую позицию относительно начала, текущего места или конца.

`ftell` сообщает текущее смещение в байтах.

`rewind` возвращает позицию в начало файла.

Текстовый и бинарный режим

Текстовый файл

удобно читать человеку и редактировать вручную

легко отлаживать на маленьких примерах

есть расходы на форматирование и разбор

Бинарный файл

компактнее и быстрее для больших массивов

неудобен для визуальной проверки

может зависеть от платформы и формата типов

Мини-упражнение на C

- Открыть file.txt, вывести первые две строки и корректно закрыть файл.
- Добавить понятное сообщение, если файл не открылся.
- Изменить режим так, чтобы программа не стирала старый лог при дозаписи.

Пути и файловая система

Разбираемся, что программа знает о
расположении файлов



Относительные и абсолютные пути

Путь к файлу — это ещё одна часть контракта между программой и средой запуска.

- Абсолютный путь указывает полный маршрут до файла в системе.
- Относительный путь считается от текущей рабочей директории процесса.
- Одна и та же программа может видеть разные файлы при разном cwd.
- Большая часть ошибок 'файл не найден' связана именно с путями, а не с `fdopen`.

Что стандартный C умеет про файловую систему

Стандарт C неплохо работает с файлами, но очень скромно говорит о каталогах и метаданных.

- `rename` и `remove` дают базовые операции над файлами.
- `tmpfile` помогает создать временный файл для промежуточной работы.
- Обход каталогов, создание папок и работа с правами обычно зависят от ОС или POSIX.

Из-за этого утилиты с путями в чистом C быстро становятся платформенно-зависимыми.

Безопасные приёмы при работе с файлами

Многие реальные ошибки связаны не с чтением байтов, а с неосторожными операциями вокруг файла.

- Не перезаписывать важный файл напрямую, если можно писать во временный и потом rename.
- Проверять, что каталог назначения существует.
- Не полагаться на то, что относительный путь 'и так должен работать'.
- Логировать имя файла и действие: open, read, write, rename, remove.

Проверка понимания перед переходом к C++

- Почему `fopen("config.txt", "r")` может не найти файл, который точно лежит рядом с проектом?
- Когда режим `w` опасен, а когда он уместен?
- Чем конец файла отличается от ошибки чтения?

Сравнение с C++

Те же задачи, но с более удобными и безопасными абстракциями



FILE* против stream-ов и RAII

C

FILE* и явный вызов fclose

fopen, fgets, fprintf, fread, fwrite

ответственность за закрытие лежит на программисте

C++

std::ifstream, std::ofstream, std::fstream

поток привязан к объекту

RAII помогает автоматизировать освобождение ресурса

Чтение файла через `std::ifstream`

В C++ поток можно проверять как объект, а строки удобно читать через `std::getline`.

```
#include <fstream>
#include <iostream>
#include <string>

std::ifstream in("report.txt");
if (!in) {
    std::cerr << "cannot open report.txt\n";
    return 1;
}
std::string line;
while (std::getline(in, line)) {
    std::cout << line << "\n";
}
```

Поток можно проверить в условии:

```
if (!in) { ... }
```

`std::string` снимает часть ограничений буфера фиксированного размера.

`std::getline` удобно сочетать с дальнейшим разбором строки.

Запись через `std::ofstream` и сборка текста

В C++ удобно сначала собрать содержимое, а затем записать его в поток.

```
#include <fstream>
#include <sstream>

std::ostringstream buffer;
buffer << "User: " << "Alice" << "\n";
buffer << "Score: " << 42 << "\n";

std::ofstream out("report.txt");
out << buffer.str();
```

`operator<<` делает запись похожей на обычный вывод в поток.

`std::ostringstream` удобен, когда нужно сначала подготовить текст целиком.

Подход хорошо сочетается с формированием отчётов и логов.

Состояния потока в C++

Поток в C++ хранит не только данные, но и своё состояние после операций ввода-вывода.

- `good()` означает, что поток пока в рабочем состоянии.
- `eof()` показывает достижение конца входного потока.
- `fail()` сигнализирует о логической ошибке чтения или форматном несоответствии.
- `bad()` обычно связан с более серьёзной ошибкой низкого уровня.

std::filesystem: базовые операции

- Начиная с C++17 стандартная библиотека умеет работать не только с файлами, но и с путями и каталогами.

`fs::exists(path)`

проверить, существует ли путь

`fs::current_path()`

получить текущую директорию

`fs::create_directory(path)`

создать каталог

`fs::remove(path)`

удалить файл или пустую папку

```
namespace fs = std::filesystem;
if (!fs::exists("output")) {
    fs::create_directory("output");
}
```

Обход каталога и работа с путями

- `std::filesystem` удобен не только для существования путей, но и для навигации по ним.

`fs::directory_iterator(path)`

обходить элементы
каталога

`entry.path().extension()`

получить расширение
файла

`entry.path().filename()`

взять только имя файла

`base / "report.txt"`

безопасно склеить путь из
частей

```
namespace fs = std::filesystem;
for (const auto &entry :
    fs::directory_iterator("logs")) {
    if (entry.path().extension() ==
        ".txt") {
        std::cout <<
            entry.path().filename() << "\n";
    }
}
```

Когда выбрать C, а когда C++

- C удобен, когда
 - нужен минимальный и предсказуемый низкоуровневый ввод-вывод
 - код учебный, встроенный или близок к системному уровню
 - задача ограничивается несколькими файлами и простым форматом
- C++ удобен, когда
 - важны строки, контейнеры и меньше ручной рутины
 - нужно обходить каталоги и работать с `path` как с отдельным типом
 - хотим безопаснее управлять ресурсами через RAII

Практическая часть: упражнения

- C: прочитать `grades.txt`, посчитать средний балл и вывести результат;
- C++: пройти по папке `data/` и вывести имена всех `.txt`-файлов;
- Написать `copy_file`: прочитать один файл и записать его содержимое в другой.
- Сделать счётчик строк, слов и символов для текстового файла.
- Для версии на C++ добавить аргументы командной строки и `std::filesystem::path`.

Типичные ошибки новичка

Большинство проблем при работе с файлами предсказуемы и хорошо лечатся аккуратной дисциплиной.

- Забыли проверить результат `fopen` или открытия `std::ifstream`.
- Перепутали режимы `r`, `w` и `a` и случайно стерли данные.
- Считали, что относительный путь считается от папки проекта, а не от `cwd` процесса.
- Перепутали конец файла с ошибкой чтения и пропустили реальную проблему.

Главные выводы

Работа с файлами — это не отдельная магия, а аккуратная работа с ресурсом, состоянием и путями.

- В C нужно явно открыть, проверить, использовать и закрыть FILE*;
- В C++ stream'ы и RAII уменьшают объём ручной рутины, но не отменяют проверку ошибок;
- Для каталогов и путей современный C++ часто удобнее благодаря std::filesystem;
- Практика важнее запоминания сигнатур: один работающий пример закрепляет тему лучше десятка определений.

Домашнее задание #4

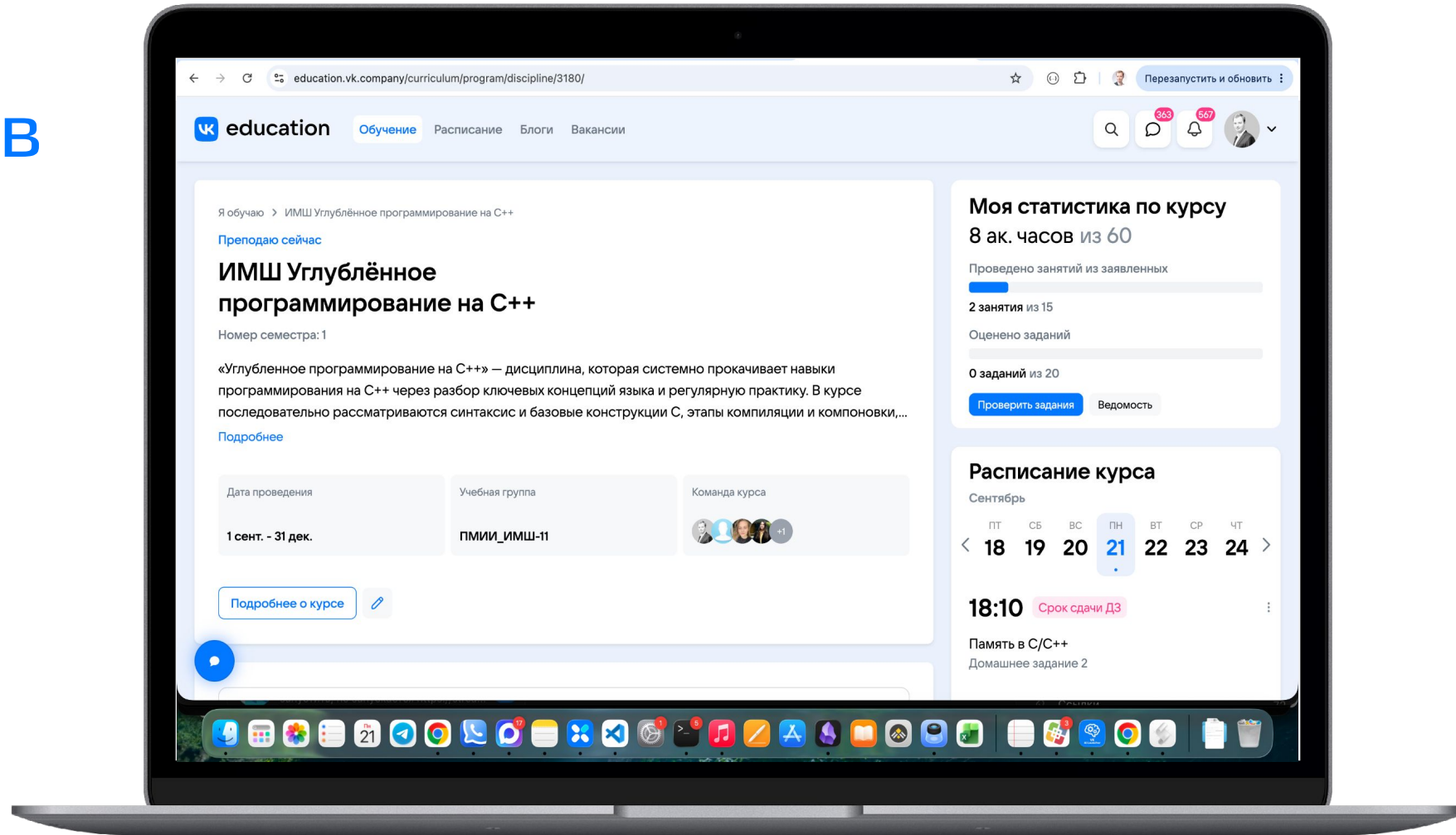
Реализовать упрощённый аналог утилиты `grep` с поддержкой флагов `-i` и `-c`.

- Программа читает указанные файлы построчно и выводит строки, содержащие заданный шаблон (подстроку);
- Флаг `-i` позволяет игнорировать регистр при поиске;
- Флаг `-c` выводит количество строк с вхождением в файле(ах);
- Не забываем проверить файл(ы) на существование!
- Реализовать наивный алгоритм поиска подстроки (но можно и Бойера-Мура и КМП при особом желании).

```
mygrep [-i] [-c] <шаблон> <файл> [<файл> ...]
```

Напоминание ОСТАВИТЬ ОТЗЫВ

Это правда важно





Спасибо
за внимание!

